

BCC202 - Estruturas de Dados I

Aula 13: Filas

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Conteúdo

Introdução

TAD Fila

Implementação via Vetor

Implementação via Ponteiros

Considerações Finais

Bibliografia

Exercício

Introdução

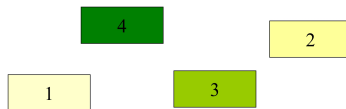
Tipo Abstrato de Dados com as seguintes características:

- ▶ O **primeiro** elemento a ser inserido é o **primeiro** a ser retirado/removido
- ▶ **FIFO - First in, first out**
- ▶ TAD conhecida como queue.

Só podemos inserir um elemento no final da fila e só podemos retirar do início.

- ▶ **Analogia** natural com o conceito de **fila** que usamos no **dia a dia**.
 - ▶ Fila bancária.
 - ▶ Fila do cinema.
- ▶ Usos:
 - ▶ Sistemas Operacionais: Fila de impressão, Fila de processamento, etc.

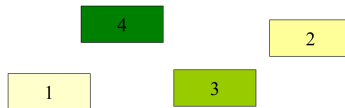
Ilustração: enfileirar e desinfileirar



Fila

Fila vazia.

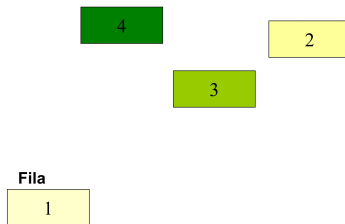
Ilustração: enfileirar e desinfileirar



Fila

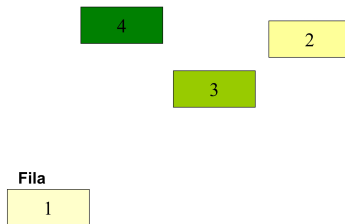
Enfileirar.

Ilustração: enfileirar e desinfileirar



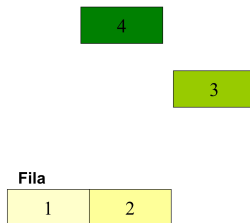
Enfileirou.

Ilustração: enfileirar e desinfileirar



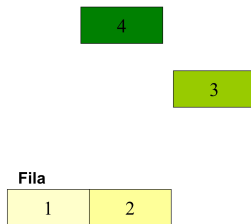
Enfileirar.

Ilustração: enfileirar e desinfileirar



Enfileirou.

Ilustração: enfileirar e desinfileirar



Enfileirar.

Ilustração: enfileirar e desinfileirar

4

Fila

1	2	3
---	---	---

Enfileirou.

Ilustração: enfileirar e desinfileirar

4

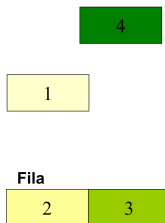
Fila

1	2	3
---	---	---

Desenfileirar.

O que é uma fila?

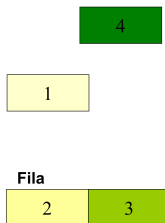
Ilustração: enfileirar e desinfileirar



Desenfileirou.

O que é uma fila?

Ilustração: enfileirar e desinfileirar



Enfileirar.

Ilustração: enfileirar e desinfileirar

4

Fila

2	3	1
---	---	---

Enfileirou.

Ilustração: enfileirar e desinfileirar

4

Fila

2	3	1
---	---	---

Enfileirar.

Ilustração: enfileirar e desinfileirar

Fila



Enfileirou.

Ilustração: enfileirar e desinfileirar

Fila

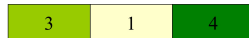


Desenfileirar.

Ilustração: enfileirar e desinfileirar

2

Fila

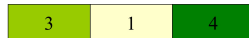


Desenfileirou.

Ilustração: enfileirar e desinfileirar

2

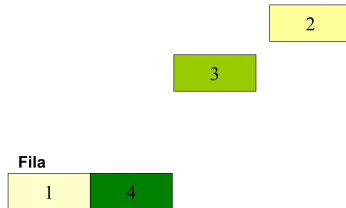
Fila



Desenfileirar.

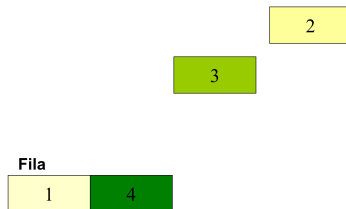
O que é uma fila?

Ilustração: enfileirar e desinfileirar



Desenfileirou.

Ilustração: enfileirar e desinfileirar



Fila nada mais é do que uma **Lista** com uma restrição:

O **primeiro** elemento a ser inserido é o **primeiro** a ser retirado.

TAD Fila

fila.h

```
1 #ifndef fila_h
2 #define fila_h
3
4 #include <stdio.h>
5
6 /*definindo um novo tipo*/
7 typedef struct { ... } Fila;
8
9 int FilaInicia(Fila*);
10 int FilaEhVazia(Fila*);
11 int FilaEnfileira(Fila*, int); /*insere no final*/
12 int FilaDesenfileira(Fila*, int*); /*retira do início */
13 void FilaLibera(Fila*);
14
15 #endif /* fila_h */
```

Existem várias opções de estruturas de dados que podem ser usadas para representar filas. As duas representações mais utilizadas são: **vetor** e **ponteiros**.

Implementação via Vetor

Vetor

Estratégia simplista:

- ▶ **Inserção** de novos elementos no **final do vetor**.
- ▶ **Retirada** de elementos no **início do vetor**.

Inserção e Remoção: Extremidades Opostas

- ▶ A operação Enfileira faz a parte de trás da fila expandir-se.
- ▶ A operação Desenfileira faz a parte da frente da fila contrair-se.
- ▶ A fila tende a caminhar pela memória do computador, ocupando espaço na parte de trás e descartando espaço na parte da frente.

Vetor

Estratégia simplista:

- ▶ **Inserção** de novos elementos no **final do vetor**.
- ▶ **Retirada** de elementos no **início do vetor**.

Inserção e Remoção: Extremidades Opostas

- ▶ A operação Enfileira faz a parte de trás da fila expandir-se.
- ▶ A operação Desenfileira faz a parte da frente da fila contrair-se.
- ▶ A fila tende a caminhar pela memória do computador, ocupando espaço na parte de trás e descartando espaço na parte da frente.

Vetor

Estratégia simplista:

- ▶ **Inserção** de novos elementos no **final do vetor**.
- ▶ **Retirada** de elementos no **início do vetor**.

Inserção e Remoção: Extremidades Opostas

- ▶ A operação Enfileira faz a parte de trás da fila expandir-se.
- ▶ A operação Desenfileira faz a parte da frente da fila contrair-se.
- ▶ A fila tende a caminhar pela memória do computador, ocupando espaço na parte de trás e descartando espaço na parte da frente.

Quais as desvantagens dessa estratégia?

Quantidade de elementos

Quais as desvantagens dessa estratégia?

Com poucas inserções e retiradas, a fila vai ao encontro do limite do espaço da memória alocado para ela.

Quantidade de elementos

Quais as desvantagens dessa estratégia?

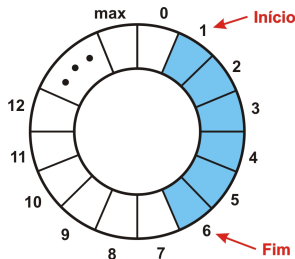
Com poucas inserções e retiradas, a fila vai ao encontro do limite do espaço da memória alocado para ela.

Quantidade de elementos

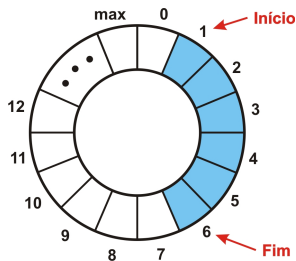
- ▶ a lista pode ter um limite conhecido (vetor estático): é necessário verificar a sua capacidade a cada inserção.
- ▶ a lista pode usar um vetor dinâmico
(dimensão atualizada – uso do *realloc* – sempre que necessário).

Fila Circular

- ▶ **Dica:** imaginar o *array* como um círculo. A primeira posição segue a última.
- ▶ Por essa característica, a fila é denominada **Fila Circular**.

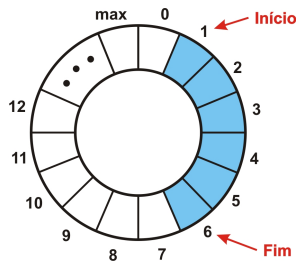


Fila Circular



- ▶ A fila ocupa posições contíguas de memória, delimitada pelos índices **Início** e **Fim**.
 - ▶ **Início** indica a posição do primeiro elemento
 - ▶ **Fim** a primeira posição vazia (posição após o último elemento)

Fila Circular



- ▶ Para enfileirar, basta mover o índice Fim uma posição no sentido horário.
- ▶ Para desenfileirar, basta mover o índice Início uma posição no sentido horário.

Fila Circular

Fila após inserção de quatro elementos

0	1	2	3	4	5	...
1.4	2.2	3.5	4.0			
↑				↑		
<i>ini</i>				<i>fim</i>		

Para essa implementação, os índices do vetor são incrementados de maneira que seus valores progridam "circularmente".

Fila Circular

Fila após retirada de dois elementos

0	1	2	3	4	5	...
1.4	2.2	3.5	4.0			
		↑		↑		
		<i>ini</i>		<i>fim</i>		

De modo geral:

- ▶ $ini = ini \% dim$, onde dim é a dimensão da fila.
- ▶ $fim = (ini + n) \% dim$, onde n é quantidade de elementos da fila e dim sua dimensão.

Struct Fila utilizando Vetores

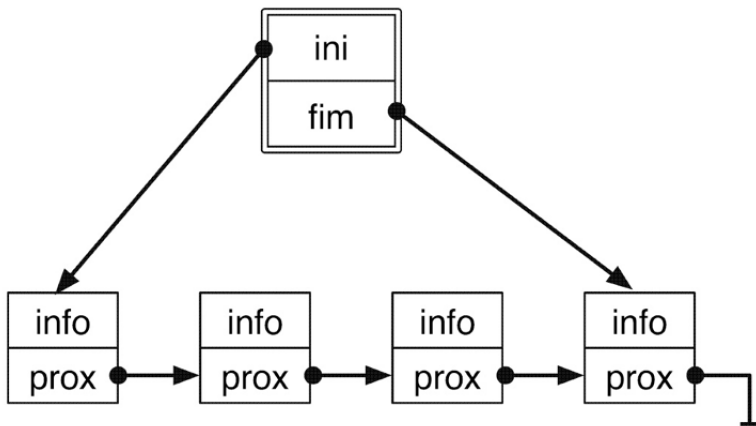
```
1  /*alt: alocar vet dinamicamente e realocar sempre que preciso. */
2  #define MAXTAM 1000
3
4  typedef struct {
5      int n;    /* número de elementos armazenados */
6      int ini; /* índice do início da fila */
7      int vet[MAXTAM]; /* vetor de elementos */
8  } Fila;
9
10 // implementação das operações de fila para manipular esses dados.
```

Implementação via Ponteiros

Filas usando ponteiros

A implementação será similar à lista encadeada com CABEÇA.

- ▶ Elementos serão inseridos e retirados de extremidades opostas da fila.
- ▶ Há uma célula cabeça para facilitar a implementação das operações Enfileira e Desenfileira quando a fila está vazia.
- ▶ Serão mantidos dois **ponteiros**: **ini** e **fim** da fila:
 - ▶ Respectivamente, **primeiro** e **último** elemento da fila.
- ▶ Quando a fila está vazia, os apontadores **primeiro** e **último** apontam para a célula cabeça.
- ▶ Para enfileirar um novo item, basta criar uma célula nova, ligá-la após a célula que contém x_n e colocar nela o novo item.
- ▶ Para desenfileirar o item x_1 , basta “desligar” a célula após a cabeça da lista.



E se mantivéssemos somente um ponteiro para a primeira posição, tal como implementamos a lista?

A especificação, é a mesma descrita para a implementação utilizando vetor.

fila_ponteiro.h

```
1  #ifndef fila_h
2  #define fila_h
3
4  /* Decisão de implementação,
   poderia ser vetor.*/
5  typedef struct {
6      int chave;
7  } Item;
8
9  typedef struct cel {
10     Item item;
11     struct cel *prox;
12 } Celula;
13
14 typedef struct {
15     Celula *cabeca;
16     Celula *fim;
17 } Fila;
18 ...
19 ...
20
21 int FilaInicia(Fila*);
22
23 int FilaEhVazia(Fila*);
24
25 /*insere no final*/
26 int FilaEnfileira(Fila*, Item);
27
28 /*retira do início */
29 int FilaDesenfileira(Fila*, Item*);
30
31 void FilaLibera(Fila*);
32
33 #endif /* fila_h */
```

O mesmo que a *Lista*: nós encadeados, contendo informações e ponteiro para o próximo nó.

fila_ponteiro.c

```
1 int FilaInicia(Fila *f) {
2     /* Considere que a fila foi declarada estaticamente na main */
3     f->cabeca = (Celula*) malloc(sizeof(Celula));
4     if (f->cabeca == NULL) return 0;
5     f->fim = f->cabeca;
6     return 1;
7 }
8
9 int FilaEhVazia(Fila *f) { return f->cabeca == f->fim; }
10
11 void FilaEnfileira(Fila *f, Item item) {
12     Celula *aux = (Celula*) malloc(sizeof(Celula));
13     aux->item = item;
14     aux->prox = NULL;
15     f->fim->prox = aux;
16     f->fim = aux;
17 }
```

fila_ponteiro.c

```
19 int FilaDesenfileira(Fila *f, Item *item) {
20     if (FilaEhVazia(f))
21         return 0;
22     Celula *aux = f->cabeca->prox;
23     f->cabeca->prox = f->cabeca->prox->prox;
24     *item = aux->item;
25     free(aux);
26     return 1;
27 }
28
29 void FilaLibera(Fila *f) {
30     /* Considere que a fila foi declarada estaticamente na main */
31     Item t;
32     while (FilaDesenfileira(f, &t));
33     free(fila->cabeca);
34 }
```

Considerações Finais

- ▶ Fila: Tipo Abstrato de Dado.
 - ▶ Vetor.
 - ▶ Ponteiro.
- ▶ **Protocolo bem definido: (First In, First Out - FIFO).**
- ▶ Qual a complexidade de tempo para enfileirar e para desinfileirar?
- ▶ Link de um exemplo de implementação.

Algoritmos de ordenação.

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados do livro-texto indicado a seguir:



Celes, Waldemar and Cerqueira, Renato and Rangel, José

Introdução a Estruturas de Dados com Técnicas de Programação em C.

Elsevier Brasil, 2016.

ISBN 978-85-352-8345-7.

Exercício

Exercício

Seja Lista o TAD Lista que implementa as seguintes funções:

```
1 /* faz com que a lista fique vazia */
2 void ListaInicia(Lista *pLista);
3
4 /* insere pItem na posicao pos da lista */
5 void ListaInsere(Lista *pLista, Item* pItem, int pos);
6
7 /* retira o item na posicao pos da lista e retorna em pItem */
8 int ListaRetira(Lista* pLista, Item* pItem, int pos);
9
10 int ListaTamanho(Lista *lista);
```

Implemente as operações abaixo do TAD Fila utilizando Lista. Esta implementação é eficiente? Justifique sua resposta.

```
1 void FilaInicia(Fila*);
2 void FilaEnfileira(Fila*, int);
3 int FilaDesenfileira(Fila*);
```